TÓPICOS ESPECIALES II / PROYECTO 1

PLAN DE TRABAJO

Equipo de tres | Aplicación base: CitaFácil

OBJETIVO

Mejorar una aplicación real de reservas con apoyo de IA y prácticas de ingeniería. La entrega se evalúa por el proceso: historias, Git, revisiones, pruebas y decisiones humanas, no por la cantidad de código generado.

Aplicación elegida

CitaFácil: FastAPI, SQLAlchemy, PostgreSQL en producción y frontend HTML/CSS/JavaScript. La copia del equipo debe vivir en un repositorio propio.

HISTORIAS QUE VAMOS A ENTREGAR

1. **Datos guardados**
   El cliente recurrente reutiliza sus datos de contacto al reservar.

2. **Calendario dinámico**
   El cliente identifica horarios libres y ocupados antes de elegir.

3. **Previsión de demanda**
   Administración consulta una estimación basada en reservas históricas.

4. **Adicional: cancelar o reprogramar**
   El cliente gestiona su cita con una verificación segura de identidad.

5. **Adicional: bloquear horarios**
   Administración bloquea fechas u horas que no deben ofrecerse.

Primer avance: previsión de demanda

Ya está implementado en la rama local feature/demand-forecast: endpoint administrativo protegido, promedio del mismo día de la semana, exclusión de reservas canceladas, gráfico y pruebas. Está pendiente la revisión del compañero Reviewer y la ejecución de QA. Sin historial, el panel debe decirlo y no aparentar una predicción confiable.

ROLES Y ROTACIÓN

Cada persona desarrolla, revisa y prueba durante el proyecto

Developer implementa y entrega una historia. Reviewer revisa el cambio y aprueba o solicita correcciones. QA prueba los criterios de aceptación y reporta evidencia. Los roles rotan cada sprint; nadie queda asignado solo a una capa técnica.

RESPONSABILIDADES EN EL SPRINT ACTUAL

Tú | Developer

Terminar y explicar la previsión de demanda. Mantener el cambio acotado, responder comentarios y no integrar antes de revisión y pruebas.

Compañero/a 2 | Reviewer

Revisar autenticación, exclusión de canceladas, media por día de semana, límites del periodo, respuesta sin historial y claridad de la gráfica.

Compañero/a 3 | QA

Ejecutar pytest y probar token ausente, historial vacío, reservas y cancelaciones; revisar el panel en escritorio y móvil, reportando pasos y resultados.

ROTACIÓN Y ENTREGAS SIGUIENTES

Sprint 1	**Tú: Developer**	**Compañero/a 2: Reviewer**	
	Compañero/a 3: QA	Previsión y gráfico	
Sprint 2	**Compañero/a 2: Developer**	**Compañero/a 3: Reviewer**	
	Tú: QA	Datos guardados y cancelación/reprogramación segura	
Sprint 3	**Compañero/a 3: Developer**	**Tú: Reviewer**	
	Compañero/a 2: QA	Calendario dinámico y bloqueo de horarios	

Reemplazar Compañero/a 2 y 3 por sus nombres antes de compartir.

CALIDAD Y PRÓXIMOS PASOS

Acuerdos para que el equipo entregue algo funcional y defendible

FLUJO OBLIGATORIO POR HISTORIA

1. Crear un Issue con historia y criterios de aceptación.
2. Analizar cómo funciona hoy el código antes de pedir cambios a la IA.
3. Crear una rama feature/<issue>-<descripcion> y hacer cambios pequeños.
4. Ejecutar pruebas automatizadas y probar manualmente los casos de aceptación.
5. Abrir PR; Reviewer y QA dejan comentarios y evidencia antes de integrar.

CRITERIOS QUE NO SE DEBEN SALTAR

- Una hora ocupada o bloqueada tampoco puede reservarse llamando la API directamente.
- No permitir reservas solapadas; las canceladas liberan disponibilidad.
- Toda acción administrativa exige token; un teléfono por sí solo no da acceso a datos personales.
- La previsión excluye cancelaciones e indica cuando falta historial.
- Validar errores de red, datos inválidos, ausencia de horarios y vista móvil.

IA y evidencia

Registrar usos significativos en AI_DEVELOPMENT_LOG.md. Documentar respuestas incorrectas y cambios rechazados solo si ocurrieron realmente: qué sugirió la IA, por qué se rechazó y cómo se verificó. Nunca incluir tokens, secretos ni datos reales de clientes.

ACCIONES INMEDIATAS DEL EQUIPO

- Asignar nombres reales y confirmar la rotación de roles.
- Crear el repositorio/fork del equipo e invitar a los tres; no subir al repositorio original.
- Instalar dependencias, ejecutar python -m pytest, revisar el cambio y abrir PR.
- Cargar datos históricos de prueba solo para la demo, nunca en producción.